

ReplayCapsule-RV: Event-Sufficient Hardware Failure Capsules for Replaying Embedded RISC-V Interrupt/MMIO Bugs

Anonymous Author(s)

ABSTRACT

Embedded firmware failures often depend on interrupt timing and memory-mapped I/O values that are visible to software but absent from ordinary crash reports. ReplayCapsule-RV records commit-indexed capsules for a scoped class of single-hart RV32I interrupt/MMIO bugs and replays those boundary events in an RTL harness. The implementation integrates a record-side hardware capsule path with PicoRV32, compiler-built RV32I firmware images, Verilator record/replay execution, an RTL capsule capture/source path, a replay-mode controller, an RTL replay consumer that drives core-facing MMIO/IRQ replay, corruption checking, runtime summaries, same-target FPGA mapping, and OpenROAD ASIC/open-PDK evidence. The generated artifact reports 45/45 v1 and 135/135 v2 compiler-backed full RTL replay PASS rows, 144/144 expanded benchmark rows across 8 additional firmware families, 243/243 captured-store self-replay handoff rows, a 42/42 focused RTL self-replay shell matrix, and a 48/48 seeded Hazard3 MMIO+IRQ replay matrix. The current prototype validates event sufficiency and replay reproducibility for bring-up/debug instrumentation with controller-driven captured-store replay evidence; it does not claim a standalone board/silicon replay engine or area-optimized shipped silicon.

1 INTRODUCTION

Embedded firmware often fails because the external world arrives at the wrong architectural moment. A sensor value crosses a threshold, an interrupt lands inside a short critical section, or a command register is sampled before firmware has restored an invariant. These failures are difficult to reproduce after the fact because the decisive facts are boundary events: interrupt delivery and return, MMIO values, and property context. Full instruction or processor-trace mechanisms can preserve far richer execution histories, but many firmware validation failures need a narrower replay object: the events that drive the same firmware-visible path to the same violation.

ReplayCapsule-RV studies that point in the design space. For a deterministic single-hart RV32I platform with the same firmware image and initial architectural state, the recorder captures commit-indexed interrupt/MMIO events, selected boundary observations, and property-failure metadata. The replay harness consumes those events at the same commit boundary and checks property metadata, event payloads, and final signatures. The design is deliberately scoped: it records replay-driving facts for a defined class of embedded failures rather than attempting to replace full trace, snapshots, or platform-wide debug infrastructure.

This paper makes three contributions:

- a commit-indexed boundary-event model for replaying scoped single-hart RV32I interrupt/MMIO failures;
- a record-side ReplayCapsule-RV hardware path plus a v2 replay-consume path that drives core-facing MMIO/IRQ replay from streamed capsule words;
- a generated evidence package covering replay success, corruption rejection, expanded benchmark breadth, baseline trace-size comparisons, runtime cost, buffer-depth tradeoffs, and same-target FPGA overhead.

The evaluation is intentionally audit-oriented. The artifact reports compiler-backed firmware builds, 45/45 v1 and 135/135 v2 full RTL replay PASS rows, full RTL corrupted-capsule checks, 144/144 expanded benchmark replay rows, runtime overhead summaries across baseline and recorder configurations, and full-core ECP5 mapped results. Claims outside this scope are treated as limitations rather than inferred results.

2 MOTIVATION

The motivating failures are firmware bugs whose manifestation depends on external ordering rather than arithmetic alone. The benchmark suite covers sensor threshold handling, interrupt races, MMIO ordering, stack protection, UART command handling, and watchdog timeout behavior. Each family has compiler-built failing and fixed firmware images; selected families also include a no-failure edge case.

The threat model is a debugging and validation setting, not an attacker model. The recorder may observe firmware-visible events and property-checker outputs. Replay starts from the same image and initial architectural state. Unsupported nondeterminism is out of scope unless it is modeled as an event. In particular, multicore interleavings, DMA, caches/coherence effects, analog device state, and undefined C behavior are not covered by the current artifact.

3 EVENT-SUFFICIENT REPLAY MODEL

The replay model is defined over architectural transitions. The original run and replay run share the same platform profile, firmware image, and initial architectural state. Between boundary events, the RV32I core semantics are deterministic. At a boundary, replay consumes the recorded event at the same commit index and validates replay-visible observations.

An event contains a commit index, a same-index order tag, a class, and a payload. Core classes are those required to drive replay in the scoped failures: MMIO read values, MMIO write observations, interrupt entry and exit, external inputs, selected stores or branch outcomes when they influence the property, check-point/hash values when used, and the property-failure signature. Diagnostic PC context is useful for debugging and comparator alignment, but the model does not assert a globally smallest trace.

Figure 1 shows the intended record/replay timeline. The recorder captures only the boundary facts needed to make the replay run take the same firmware-visible path through the target



Figure 1: Record/replay timeline with commit-indexed events. Source generated from `scripts/make_figures.py`.

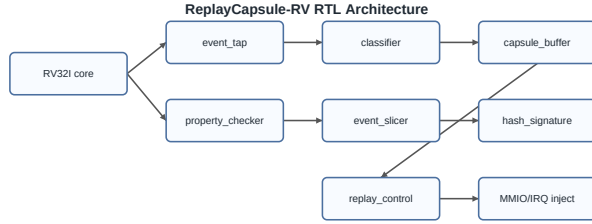


Figure 2: ReplayCapsule hardware architecture. Source generated from `scripts/make_figures.py`.

failure prefix. Overflow invalidates replay availability; it is a status bit, not a recoverable compression mode.

The formal replay-sufficiency sketch is by induction over commit index: equal initial state, deterministic ordinary steps, equal boundary-event payloads at nondeterministic steps, and deterministic property checking over the equal prefix.

4 HARDWARE ARCHITECTURE

ReplayCapsule-RV instruments the processor boundary rather than replacing the core. The PicoRV32 wrapper exposes memory, MMIO, interrupt, and commit-context signals to the record-side modules. The event tap observes candidate facts, the classifier selects replay-relevant and diagnostic classes, the slicer retains context around a property failure, the capsule buffer stores encoded packets, and the property checker freezes the capsule on a target violation.

The board-level mapped wrappers expose only clock, reset, UART, GPIO, status LEDs, and three recorder-status pins. Large debug and internal buses remain inside the FPGA design. A v1 mapped build disables redundant v2 recorder instances that are not selected by the architecture parameter, but the remaining recorder, event packet storage, signature path, property checker, and status visibility still cost substantial resources. The current same-target ECP5 rows report LUT overhead from 114.20% to 221.23%, FF overhead from 174.53% to 646.43%, and Fmax change from -19.56% to -9.98%. This overhead is acceptable for the intended bring-up/debug-instrumentation use case, where fidelity and auditability matter more than shipped-silicon area; it is not presented as an optimized always-on production block.

The selected v2 minimal recorder profile is a separate compile-time mapped path. It removes the property checker, context slicer, on-chip event buffer, hash signature, and detailed counters from the recorder instance, while still emitting commit-indexed MMIO read/write, external-input, and interrupt enter/exit events for host

capture. On the same ECP5 point, that profile maps at 2818 LUT, 992 FF, 3 BRAM, and 68.82 MHz against a 2603 LUT, 956 FF, 3 BRAM, and 68.85 MHz baseline. The core and hashed v2 rows remain diagnostic-rich comparison configurations.

4.1 ReplayCapsule-RV v2 Prototype

ReplayCapsule-RV v2 keeps the v1 recorder intact and adds separately selectable compressed recorder profiles. The diagnostic v2 path packs replay-critical event class, commit-delta, address-token, payload-hash, and limited debug context into 64-bit capsule words; repeated MMIO addresses can use a small dictionary, and diagnostic context can be dropped under adaptive buffer pressure while replay-critical MMIO, interrupt, property-fail, and checkpoint-hash events remain capturable. The selected minimal recorder profile keeps only the replay-driving boundary-event stream. The v2 implementation also includes a synthesizable replay-consume controller prototype that validates event type, commit order, MMIO value, interrupt cause, and payload hash before reporting that all capsule events were consumed.

The v2 recorder and replay consumer are not presented as a standalone board/silicon replay engine. In this artifact, v2 recorder RTL is selectable in the full-core record path, the RTL source has a capture-store path that retains replay-critical words across reset in directed HDL tests and v2 record signatures, and a replay-mode controller now clears/arms that store and starts captured-store replay from the retained count. The replay-consume controller has directed RTL corruption tests, saved-capsule full-core checks, and same-instance self-replay checks in which the replay run streams from the captured RTL store without harness-preloaded capsule words. During replay, that source streams the capsule while the consumer validates observed boundary events and drives core-facing MMIO read data and IRQ replay through the PicoRV32 wrapper. The remaining missing piece is a standalone board/silicon replay flow outside the Verilator reset orchestration and memory/peripheral shell.

5 RECORD/REPLAY IMPLEMENTATION

The replay comparator consumes an expected capsule and an observed replay trace. A passing comparison requires matching property metadata, the required event classes at the selected commit or cycle index, exact payload equality for replay-driving events, and no extra replay-driving events. Negative tests corrupt metadata, remove events, duplicate events, shift indices, swap order tags, and perturb payloads. The same comparator is used for model fixtures and exported RTL-smoke capsules.

The current replay-consume path is hardware-driven at the capsule stream and MMIO/IRQ boundaries. The saved-capsule replay ledger records a capsule from the instrumented PicoRV32 wrapper, reruns the same compiler-backed firmware under replay control, preloads the replay-critical capsule words into an RTL capsule source, and checks the resulting trace. The self-replay ledger exercises the stronger handoff: the same Verilator instance records replay-critical words into the RTL capture store, resets the core while retaining that store, and uses an RTL replay-mode controller to launch replay from the captured store without harness-preloaded capsule words. During v2 replay, the host no

longer streams capsule words cycle by cycle and no longer computes replay MMIO values or IRQ timing; the RTL replay-mode controller selects the captured source, the RTL source streams capsule words into the RTL consumer, and the consumer validates the observed event sequence while driving core-facing MMIO read data and IRQ replay through the PicoRV32 wrapper. This is stronger evidence for full-core replay plumbing, but it is not a standalone replay engine because reset orchestration and the memory/peripheral shell remain in the Verilator harness.

Bounded local checks cover recorder and control contracts such as event classification, buffer behavior, property signatures, replay mismatch handling, register access, hash/signature update, and MMIO/interrupt logging. These checks support local RTL obligations; they are not a mechanized end-to-end processor theorem.

6 EVALUATION

The evaluation is generated from repository scripts and processed CSVs; the paper-number audit rejects numeric claims that do not appear in generated evidence. The core full-RTL ledger reports 45/45 compiler-backed record/replay PASS rows. The negative ledger rejects 10 replay-critical corruptions, reports 0 unexpected accepts, and leaves 2 not-applicable rows for corruption classes outside the current commit-index harness.

6.1 Replay, Scale, and Buffers

Replay succeeds on the fixed compiler-backed suite in the full-RTL harness, but scaling is constrained by capsule capacity. The re-generated workload-scaling evidence reports 375/375 PASS rows with workload-aware v1 depths: 256 for smoke and short workloads, 1024 for medium, 4096 for long, and 16384 for stress. The buffer-sensitivity table shows replay success reaches 100.000000% for each workload class at those selected depths, while the sticky overflow bit remains asserted on long-running fixed and no-failure rows that do not freeze at a property failure. The paper therefore treats buffer depth as a design parameter, not a hidden constant.

Capsule-size comparisons use the buffer depth required for the workload class under test. `capsule_baseline_summary.csv` reports median ReplayCapsule reductions versus the raw full instruction trace estimate of 59.52% for smoke, 78.71% for short, 83.18% for medium, 64.86% for long, and 22.84% for stress. The measured v2 baseline summary adds a harder stress-row comparison: full instruction traces have median 445904.000000 bytes and full commit traces have median 891808.000000 bytes, while the v2 core and hashed capsule rows have median 16.000000 and 16.000000 bytes. The same CSV includes a labeled `riscv_etrace_branch_trace_estimate` row; that estimate is smaller than ReplayCapsule-RV for these workloads, so the raw instruction trace comparison is a weak baseline, not a state-of-the-art trace-compression claim.

The v2 evidence package adds compressed recorder configurations and 8 expanded firmware families covering commanded actuator limits, late configuration sequencing, sensor debouncing, clear-on-read command/status behavior, alternate-MMIO-profile command/configuration cases, and realistic environmental-controller/power-rail control cases. The expanded replay ledger

reports 144/144 PASS rows across core, hashed, and full recorder configurations with an RTL capsule source and RTL-driven MMIO/IRQ replay stimulus. The self-replay handoff ledger further reports 243/243 replay-mode-controller rows across the base and expanded benchmark families, showing that replay-critical words captured during record can drive same-instance replay without harness-preloaded capsule words. These rows improve workload and platform-profile coverage for the scoped model, but they do not replace the Verilator reset orchestration or memory/peripheral shell with a standalone board/silicon flow.

6.2 Cost and Mapping

The mapped FPGA results are same-target bring-up/debug measurements. The board-level baseline and ReplayCapsule wrappers use the same `ecp5-85k` target and `yosys+synth_ecp5+nextpnr-ecp5` flow. After gating unselected v2 recorder instances in v1 mapped builds, the representative mapped-scaling summary reports LUT overhead from 114.20% to 221.23%, FF overhead from 174.53% to 646.43%, BRAM overhead of 0.00%, and Fmax change from -19.56% to -9.98%.

For the selected v2 minimal recorder profile, the same-target ECP5 measured row reports +8.26% LUT, +3.77% FF, +0.00% BRAM, and -0.04% Fmax. The core and hashed v2 rows remain diagnostic-rich comparison points, with +68.00% and +67.69% LUT overhead and +81.49% FF overhead. These rows support the design direction that replay-critical boundary-event capture should be separated from optional on-chip diagnostics.

The artifact also adds a Nangate45 ASIC/open-PDK lane. The Yosys+ABC synthesis-only rows report baseline cell area of 51157.652 square micrometers and selected v2 minimal area of 52058.062 square micrometers, or +1.76%. The OpenROAD physical-flow rows place the same baseline and v2 configurations and run global routing with parsed area, WNS/TNS, and power. Baseline and selected v2 minimal report 53453.000 and 54374.000 square micrometers, respectively, both with 0.000 ns WNS and 5.990000 mW versus 6.590000 mW total power. These rows are global-routed implementation evidence, not detailed-route signoff or silicon energy data.

This overhead is acceptable only under the intended use case: temporary validation and post-failure replay instrumentation during hardware/software bring-up. It is not presented as a production always-on block. FPGA and Nangate45 results should be read as feasibility and cost-shape evidence rather than signoff implementation data.

7 RELATED WORK

RISC-V E-Trace, N-Trace, and external debug specifications target processor observability, control-flow reconstruction, and standardized debug transport [6, 7]. ReplayCapsule-RV is complementary: it records commit-indexed replay-driving interrupt/MMIO events for a scoped single-hart RV32I failure class. It does not attempt to provide architectural trace coverage or debug interoperability.

Post-silicon trace compression reduces trace bandwidth and storage by exploiting structure in instruction or control-flow streams. ReplayCapsule-RV makes a different tradeoff. It records

the boundary events needed by the replay harness, not a compressed representation of the full executed instruction stream.

Deterministic replay systems record nondeterministic inputs and ordering so an execution can be reconstructed [1, 4]. ReplayCapsule-RV borrows that replay discipline but narrows the target to embedded interrupt/MMIO failures, uses commit-indexed hardware capsules, and evaluates a PicoRV32 RTL harness rather than an operating-system or virtual-machine replay substrate.

Runtime verification and assertion monitors detect violations close to the hardware/software boundary [5]. ReplayCapsule-RV uses property checking as the capsule trigger, but its contribution is the replay-driving event stream and the evidence that corrupted capsules fail comparison.

Embedded event logs are widely used to explain firmware failures after deployment. ReplayCapsule-RV differs by making event ordering architectural through commit indices and by validating the capsule through replay comparison instead of presenting the log only as diagnostic text.

FPGA/emulation debugging and checkpointing can capture broad hardware state and rerun workloads under controlled conditions. Snapshot and time-travel systems similarly preserve enough state to resume or inspect execution [3]. ReplayCapsule-RV is narrower: it keeps the same initial state and firmware image, then records the boundary events needed for a selected failure prefix.

The side-by-side distinction is:

- Full instruction or commit traces capture the retired stream or per-commit state; ReplayCapsule-RV records replay-driving boundary events and property-fail context.
- RISC-V E-Trace and N-Trace-style mechanisms capture compressed control-flow discontinuities and synchronization; the generated E-Trace-style estimate is a stronger size baseline than raw instruction trace, while ReplayCapsule-RV carries MMIO values and interrupt timing for replay.
- Deterministic VM replay captures device inputs and scheduling at OS or VM granularity; ReplayCapsule-RV uses commit-indexed single-hart RV32I boundary events inside a firmware-running RTL harness.
- Runtime-verification monitors capture assertion outcomes and checker state; ReplayCapsule-RV uses the checker as a trigger and validates the captured event stream by record/replay comparison.

In the current implementation, v1 packets are 168.000000-bit boundary-event records and v2 measured rows use compact 64-bit exported capsule words where integrated; the claim is scoped replay sufficiency for interrupt/MMIO bugs, not trace-standard replacement or autonomous full-core replay consume.

The implementation builds on PicoRV32, Verilator, Icarus Verilog, Yosys, nextpnr, and bounded formal tooling [2, 8–12]. These tools support reproducibility; the claimed research object is the scoped capsule model, recorder, replay harness, and evidence package.

8 LIMITATIONS

The scope is single-hart RV32I firmware with deterministic core semantics, the same program image, the same initial architectural state, and modeled external events. The current theorem excludes

multicore interleavings, DMA unless modeled, cache/coherence effects unless modeled, analog nondeterminism, and claims about undefined C behavior.

MMIO side effects are modeled only at the replay-visible boundary. Interrupts are injected at an architectural boundary. Replay consumes every required event exactly once; missing, duplicate, reordered, or corrupted events must fail comparison. Capsule overflow invalidates replay availability for failure reconstruction unless the replay-critical prefix is still complete.

The v2 replay-consume path now includes same-instance self-replay rows that stream from the captured RTL store without harness-preloaded capsule words, and an RTL replay-mode controller now sequences captured-store arming and replay start. The path is still orchestrated by the Verilator harness for reset control and the memory/peripheral shell. The v2 full-core simulation path includes a synthesizable capture/source path and consumer: the record path exercises reset-persistent capture of replay-critical words, the source streams capsule words, and the consumer drives core-facing MMIO read data and IRQ replay. The design does not yet include a standalone board/silicon replay flow. The v1 workload evidence recovers 375/375 replay PASS rows only by using workload-aware depths: 256 for smoke and short, 1024 for medium, 4096 for long, and 16384 for stress. These depths increase capsule bytes and reduce the median stress-workload reduction versus a raw full-instruction trace to 22.84%. High FF/LUT overhead remains an optimization target for diagnostic-rich configurations: same-target ECP5 overhead is still as high as 221.23% LUT and 646.43% FF in the broader mapped-scaling rows, and the v2 core/hashed diagnostic rows remain at +68.00%/+67.69% LUT and +81.49% FF. The selected v2 minimal recorder profile is measured separately. Nangate45 ASIC rows use OpenROAD placement and global routing with parsed timing and power, but they are not detailed-route signoff, tapeout, silicon, or energy measurements. The evaluation is single-hart RV32I and does not cover multicore, DMA, caches/coherence, or arbitrary peripherals.

ReplayCapsule-RV v2 reduces the architectural risk by adding compressed recorder RTL, an RTL capsule capture/source path, a replay-mode controller, and a synthesizable replay-consume controller. The controllers have tested stream consumption, order checking, MMIO/interrupt checking, payload-hash rejection, saved-capsule full-core checks, and controller-driven captured-store self-replay checks across the measured v2 rows. The replay consumer also maps as a standalone ECP5 block. These rows should not be used to claim a standalone board/silicon replay engine, because reset orchestration and the memory/peripheral shell remain in the harness.

9 CONCLUSION

ReplayCapsule-RV shows that a scoped class of embedded RV32I interrupt/MMIO failures can be recorded as commit-indexed boundary-event capsules and replayed in an RTL harness with compiler-backed firmware. The artifact reports 45/45 v1 full RTL replay rows, 135/135 v2 rows with RTL capsule-source and MMIO/IRQ replay stimulus, replay-mode-controller captured-store self-replay handoff evidence, corruption rejection, expanded benchmark coverage, runtime summaries, buffer-depth tradeoffs,

hard trace-size baselines, same-target FPGA overhead, and Nan-gate45 OpenROAD global-routed area/timing/power rows. The result is not a replacement for trace standards or snapshots; it is a hardware capsule design and evidence package for replay-driving boundary events. The next step is detailed ASIC signoff, standalone replay flow beyond Verilator reset/peripheral orchestration, and broader platform modeling, not stronger claims from the current prototype.

REFERENCES

- [1] George W. Dunlap, Samuel T. King, Sukru Cinar, Murtaza A. Basrai, and Peter M. Chen. 2002. ReVirt: Enabling Intrusion Analysis through Virtual-Machine Logging and Replay. In *OSDI*.
- [2] Icarus Verilog Project. 2026. Icarus Verilog. <https://steveicarus.github.io/iverilog/>.
- [3] Samuel T. King, George W. Dunlap, and Peter M. Chen. 2005. Debugging Operating Systems with Time-Traveling Virtual Machines. In *USENIX Annual Technical Conference*.
- [4] Thomas J. LeBlanc and John M. Mellor-Crummey. 1987. Debugging Parallel Programs with Instant Replay. In *IEEE Transactions on Computers*.
- [5] Martin Leucker and Christian Schallhart (Eds.). 2009. *Runtime Verification*. Springer.
- [6] RISC-V International. 2026. RISC-V External Debug Support. <https://github.com/riscv/riscv-debug-spec>. Accessed 2026-06-26.
- [7] RISC-V International. 2026. RISC-V Processor Trace Specification. <https://github.com/riscv-non-isa/riscv-trace-spec>. Accessed 2026-06-26.
- [8] Verilator Project. 2026. Verilator. <https://verilator.org/>.
- [9] Clifford Wolf. 2026. PicoRV32: A Size-Optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>.
- [10] YosysHQ. 2026. nextpnr: portable FPGA place and route. <https://github.com/YosysHQ/nextpnr>.
- [11] YosysHQ. 2026. SymbiYosys and Yosys SMTBMC. <https://yosyshq.readthedocs.io/projects/sby/>.
- [12] YosysHQ. 2026. Yosys Open SYnthesis Suite. <https://yosyshq.net/yosys/>.